AI Compiler Performance Engineering

Auto-Loops Books — ch10

Contents

1 Attention Online Softmax Optimization **1**

1.1 Online Softmax 2

1.2 Numerical Stability 7

1.3 Cross-Hardware Online Softmax 9

1.4 Softmax Fusion Boundaries 11

1.5 Profiling Online Softmax in Production Stacks 17

1.6 Key Takeaways 20

1.7 Conclusion 23

Bibliography **25**

Chapter 1

Attention Online Softmax Optimization

Can software alone recover decode when interconnect and HBM budgets are tight? Fregly's DeepSeek case study shows that mechanical sympathy—algorithms written for the memory hierarchy they run on—can match frontier quality on constrained H800 clusters (Liu *et al.*, 2024; Davies *et al.*, 2025). The same principle applies at batch-1 decode inside a single layer: naive attention materializes an $S \times S$ softmax map, and even a streaming $1 \times T$ score row becomes catastrophic when fused boundaries force write/read pairs between KV blocks (Dao *et al.*, 2022, 2023; Chen, 2026).

Online softmax is the attention-side embodiment of that sympathy: instead of storing $P = \text{softmax}(QK^\top)$, the kernel maintains running statistics (m, ℓ) (row max and sum of exponentials) and a partial value accumulator o, updating all three as each KV block arrives (Dao *et al.*, 2022). FlashAttention made IO-aware tiling mainstream; Flash-Decoding parallelizes the KV dimension while preserving online state (Dao *et al.*, 2023). This chapter explains the numerics, residency tables on GPU/XDNA/CPU, and the **fusion boundaries** where online carriers must hand off to PV and Chapter 11 MegaKernels without touching HBM (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020).

The common production mistake is treating softmax as a cheap epilogue after a GEMM. At batch-1 decode, softmax is not one kernel—it is a *state machine* embedded inside the KV loop (Kwon *et al.*, 2023; Davies *et al.*, 2025). TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; online softmax alone will not collapse that count—fusion with QKV and PV inside a MegaKernel will (Chapter 11) (Vellaisamy *et al.*, 2026; NVIDIA, 2024). But

MegaKernels *require* online carriers to stay in registers or shared memory across the attention–MLP seam (SemiAnalysis, 2024; AMD, 2024a). Spill (m, ℓ, o) between blocks and you pay full-width traffic every context step while launches/token flatline (Chen, 2026).

Decode vs prefill softmax shape. Prefill processes many query rows at once; decode adds *one* new query row against entire KV history (Dao *et al.*, 2023; Kwon *et al.*, 2023). Online softmax in decode is a loop over KV *blocks*, not a single matmul epilogue (Dao *et al.*, 2022). Benchmarks that only tune prefill softmax miss the carrier residency story that determines whether Chapter 11 fusion is even legal (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Training vs inference scope. Online softmax also accelerates training attention for long sequences—FlashAttentionś original motivation (Dao *et al.*, 2022). This chapter emphasizes decode because batch-1 residency and MegaKernel fusion are decode-specific engineering pressures (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023; Davies *et al.*, 2025). Training chapters in other books cover overlapping comm/compute; our decode compiler story stands alone (Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026).

Goodput for attention. Raw FLOPs on $QK^\top$ mislead: useful decode work is the final attention output bytes delivered per ms/token, discounting materialized P, redundant HBM round trips, and extra launches (Davies *et al.*, 2025; Chen, 2026). Online softmax raises attention goodput by keeping exponentials and partial PV on-chip—the attention analog of Meta's goodput metric for training clusters (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026).

1.1 Online Softmax

Step-by-step decode loop (one head). At token t, the query vector q is fixed. For each KV block $b = 1 \dots B$:

1. TMA or vector loads fetch K_b, V_b tiles into on-chip storage (SemiAnalysis, 2024; Kwon *et al.*, 2023).
2. MMA or dot products form score tile s_b (NVIDIA, 2022; Dao *et al.*, 2022).
3. Warp max-reduce finds m_b; threads compute $e^{s_b - m'_b}$ with updated global max m'_b (Dao *et al.*, 2022).

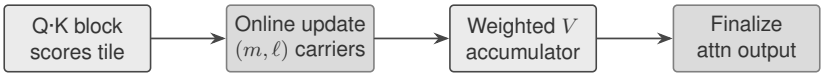


Figure 1.1: Online softmax pipeline: scores enter, carriers and partial PV stay on-chip across KV blocks.

4. Accumulate ℓ_b and $o_b = \sum \text{softmax}(s_b)\, V_b$ (Dao *et al.*, 2022, 2023).

5. Merge into running (m, ℓ, o) without global memory (Chen and YiRage Contributors, 2026; Chen, 2026).

Finalize o/ℓ once after block B (Goodfellow *et al.*, 2016). This loop is the device-side spec Chapter 11 MegaKernels implement (Vellaisamy *et al.*, 2026; Chen, 2020).

Bytes/token estimate (sketch). Materializing P adds roughly $2T \cdot d_{\text{head}}$ bytes read/write per head per layer per token (scores + probabilities), ignoring cache effects (Chen, 2026; Davies *et al.*, 2025). Online softmax removes the P term; remaining traffic is dominated by KV and weights (Kwon *et al.*, 2023; Dao *et al.*, 2022). Use this sketch to justify fusion PRs when microbench GEMMs look unchanged (Vellaisamy *et al.*, 2026).

Comparison to SDPA eager paths. PyTorch scaled dot-product attention may call cuDNN or flash backends depending on shape (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). At batch-1 decode on bandwidth-saturated GPUs, swapping SDPA variants without fixing launches/bytes can *regress* latency (Chen, 2026; Vellaisamy *et al.*, 2026). Online softmax must be evaluated inside full-layer counters, matching Fregly's profile-first methodology (Davies *et al.*, 2025).

The naive failure mode. Standard attention computes scores $s_{ij} = q_i \cdot k_j / \sqrt{d_h}$, applies softmax row-wise, multiplies by V (Goodfellow *et al.*, 2016; Dao *et al.*, 2022). Materializing P for decode means an $1 \times T$ vector per head per layer per token—small individually, catastrophic when each block boundary exports scores or probabilities to HBM (Vellaisamy *et al.*, 2026; Chen, 2026). Online softmax avoids storing P entirely (Dao *et al.*, 2022).

For block t with local scores $x_j^{(t)}$, maintain running maximum m, sum $\ell = \sum \exp(x_j - m)$ across blocks seen so far, and accumulator $o = \sum_j \text{softmax}(x)_j v_j$ (Dao *et al.*, 2022). When a new block arrives with local max m_t and local sum

Table 1.1: Materialized softmax vs. online softmax (decode, one head).

Aspect	**Materialized P**	**Online carriers**
Extra HBM for P	$O(T)$ per head	$O(1)$ statistics
Launches if split out	+1 per block risk	In KV loop body
Carrier residency	N/A	Registers / shared mem
MegaKernel legality	Poor	Required
XDNA static plan	Often illegal	Assign (m, ℓ, o) slots

$\ell_t = \sum_j e^{x_j^{(t)} - m_t}$, update:

$$
\begin{aligned}
m' &= \max(m, m_t), \\
\ell' &= \ell \cdot e^{m - m'} + \ell_t \cdot e^{m_t - m'}, \\
o' &= o \cdot e^{m - m'} + o_t \cdot e^{m_t - m'}.
\end{aligned}
$$

After the final block, o/ℓ is the attention output row (Dao *et al.*, 2022, 2023). Figure 1.1 is the control-flow spine every fused decode kernel must implement (Chen and YiRage Contributors, 2026; Chen, 2020).

Log-sum-exp view. Equivalently, online softmax tracks $\log \ell$ and m so merges use stable log-add operations (Goodfellow *et al.*, 2016). Compiler backends may emit either multiplicative or log-space updates; IR must preserve associativity order for deterministic mode (Chen and YiRage Contributors, 2026; Chen, 2020). Mixing orders across GPU warps without documentation breaks bitwise regression tests (Davies *et al.*, 2025).

Multi-head carrier layout. With h heads, each head owns (m_h, ℓ_h, o_h) or shares warp groups (Dao *et al.*, 2022). MegaKernels pack carriers in shared memory with head-major or block-major layouts depending on MMA tile shapes (SemiAnalysis, 2024; NVIDIA, 2022). XDNA assigns fixed SRAM slots per head at compile time—dynamic head counts require separate compiled graphs (AMD, 2024a,b).

Table 1.1 explains why "online" is a **residency contract**, not an algorithmic nicety (Chen, 2026; Davies *et al.*, 2025). GPU implementations keep (m, ℓ, o) in warp registers or shared memory for the duration of the KV block loop (SemiAnalysis, 2024; NVIDIA, 2022). Exporting carriers between blocks to global memory doubles traffic and breaks Chapter 11 fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Paged KV interaction. vLLM-style block tables change *where* K/V bytes live, not whether online updates run each block (Kwon *et al.*, 2023). Device-side pointer walks replace scatter kernels but the inner softmax state machine is unchanged (Davies *et al.*, 2025). Block size interacts with tile shapes: 16-token blocks may underutilize bandwidth; 256-token blocks may exceed shared-memory staging (Dao *et al.*, 2023; SemiAnalysis, 2024).

Example 1.1.1. Decode cell (BS=1, ctx=2048, block=64): 32 KV blocks per head. Materialized softmax writes 32 partial P tiles to HBM; online softmax keeps three scalars/vectors per warp group on-chip (Dao *et al.*, 2022; Chen, 2026; Vellaisamy *et al.*, 2026).

1.1.1 KV block and split sizing

Choosing how many KV positions one online step consumes is a joint constraint problem, not a fixed template. Each step needs one score tile, one K tile, one V tile, and room for the carriers; the product of head dimension, bytes per element, and the two KV copies determines how many bytes must arrive per step, while the on-chip budget decides whether a larger block or a second split is the better fit (Chen, 2026; Dao *et al.*, 2022).

On Hopper-class GPUs the sizing loop starts from the memory system: pick a candidate block, compute its K/V staging against shared memory, subtract the score tile and carrier footprint, and accept the largest block that leaves a full double-buffer slot for the next tile (SemiAnalysis, 2024; NVIDIA, 2022). Blocks that undershoot the transfer granularity leave bandwidth idle; blocks that overshoot spill score tiles or force carrier export. Either failure shows up as bytes/token drift before latency does, so the worksheet must record the chosen block size next to the measured counters (Chen, 2026; Vellaisamy *et al.*, 2026).

The sizing question has a second axis on parallel hardware: when one block would exceed per-warpgroup staging, split the KV range across groups and merge with the online equations from this section. Use the smallest split count that satisfies residency, because every additional split adds a merge step and a synchronization point (Dao *et al.*, 2023; Davies *et al.*, 2025). XDNA compiles the same choice into a static DMA schedule, so a block size that needs dynamic spill fails compile and makes the constraint visible earlier than on GPU (AMD, 2024a,b). CPU sizing replaces the shared-memory budget with L1/L2 cache lines and replaces split synchronization with SIMD lanes, but the ordering—bytes per step, staging budget, then merge cost—stays identical (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

FlashAttention and Flash-Decoding. FlashAttention uses online softmax inside SRAM tiles for prefill-like parallelism (Dao *et al.*, 2022). Flash-Decoding splits KV across SMs while preserving online state per split, then merges splits with a second online reduction (Dao *et al.*, 2023). Decode MegaKernels must choose split strategy *inside* one launch or pay multiplied graph steps (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Split-merge is easy to get wrong: an extra global sync per split negates Flash-Decoding wins (Davies *et al.*, 2025).

Split-KV merge protocol. When SM s finishes block range $[b_s, b_e)$ with local (m_s, ℓ_s, o_s), a tree reduction combines pairs using the same merge equations until one global (m, ℓ, o) remains (Dao *et al.*, 2023). Implementations either use a dedicated merge kernel (bad for launches/token) or warp-specialized epilogue warps inside the MegaKernel (preferred) (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Profile merge separately—it becomes visible at long context when split count grows (Davies *et al.*, 2025).

GQA and MQA carrier savings. Grouped-query attention shares K/V across query heads, reducing KV bytes/token but not eliminating online softmax per query group (Liu *et al.*, 2024; Kwon *et al.*, 2023). Carrier count scales with query groups, not raw head count—compiler passes must not allocate per-head carriers when groups share scores (Chen and YiRage Contributors, 2026; Chen, 2020).

Carrier reset and query-row layout. Every decode token begins a fresh online softmax over the current KV history, so the carrier state resets once per query row, not once per kernel launch (Dao *et al.*, 2022; Kwon *et al.*, 2023). In batch-1 decode that reset is a register write before the first KV block; in chunked prefill or continuous batching, several query rows share the same KV pass and carriers must be laid out row-major so each rowś (m, ℓ, o) merges independently (Dao *et al.*, 2023; Davies *et al.*, 2025). A bug that reuses the previous tokenś ℓ across steps is invisible in unit tests that run a single query but produces drifting attention weights in production, so reset behavior belongs in the same IR lifetime analysis as the carriers themselves (Chen and YiRage Contributors, 2026; Chen, 2020). Treat the reset as an explicit SSA initialization in lowering; a pass that hoists it out of the per-token loop is changing model semantics, not optimizing it (Chen, 2026).

Recording the sizing decision. Block size, split count, and carrier layout are performance decisions with correctness impact: changing any of them changes the bytes/token worksheet and the merge tree that stability tests must cover

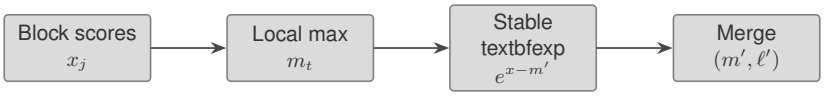


Figure 1.2: Numerical stability path: max-find precedes exponentials; merges preserve online invariants.

(Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Store them as IR attributes on the online softmax region so a later refactor cannot alter residency silently (Chen, 2020, 2026). Code review should show the attributes changing when the kernel changes, which keeps autotuners honest about what they are actually searching (Davies *et al.*, 2025).

1.2 Numerical Stability

Error propagation across merges. Each merge is multiplicatively stable when m tracks the true row max (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). If local max is underestimated, exponentials overflow and ℓ corrupts silently (Davies *et al.*, 2025). Tests must include adversarial score spikes at random positions within long blocks (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Determinism vs performance. Parallel max reductions across warps choose associativity order (NVIDIA, 2022; Chen, 2020). Compliance workloads may require deterministic reductions; interactive serving often tolerates bounded drift (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Document reduction order in kernel headers and YiRage lowering logs (Chen and YiRage Contributors, 2026).

Interaction with rotary embeddings. RoPE rotates q, k before $QK^\top$; it does not change softmax merge math but shifts score distributions with position (Liu *et al.*, 2024; Kwon *et al.*, 2023). Long-context stability tests must use RoPE-enabled references, not static q, k (Dao *et al.*, 2023; Davies *et al.*, 2025).

Why max-subtraction is non-optional. Softmax exponentials overflow in FP16/BF16 without subtracting row max (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Online softmax *is* the stable formulation—but only if each block computes local max before **exp** (Dao *et al.*, 2022). A "fast path" that skips max on "small" blocks fails at long contexts when score magnitudes drift with RoPE position (Dao *et al.*, 2023; Chen, 2026).

Table 1.2: Carrier dtype choices (engineering guidance).

Carrier	**FP16**	**BF16**	**FP32**
m (max)	Risky	OK	Best
ℓ (sum exp)	Underflow risk	OK	Best
o (value acc)	Rarely sufficient	Often OK	Best

Figure 1.2 is the micro-pipeline inside each KV block (Dao *et al.*, 2022). **Stable** implementations fuse max-reduce and scaled **exp** in shared memory to avoid an extra global round trip (SemiAnalysis, 2024; NVIDIA, 2022). Hopper SFUs accelerate **exp** relative to prior generations—mechanical sympathy at the silicon level for transformer attention (NVIDIA, 2022; Davies *et al.*, 2025).

FP8 and block scales. FP8 decode experiments carry per-block or per-tensor scales into the online merge (Davies *et al.*, 2025; Liu *et al.*, 2024). Re-quantizing scores in a separate kernel breaks fusion and often breaks stability unless scales participate in the max-find (Chen and YiRage Contributors, 2026; Chen, 2020). Treat scale vectors like carrier extensions in IR lifetime analysis (Chen, 2026).

Precision pitfalls. Using FP16 for ℓ while FP32 for m is common; using FP16 for both can underflow ℓ at long T (Dao *et al.*, 2022; Davies *et al.*, 2025). BF16 often behaves better for decode carriers but still needs FP32 accumulation for o (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). Compiler passes should declare accumulator dtypes explicitly (Chen, 2020; Chen and YiRage Contributors, 2026).

Testing stability. Compare online vs reference double-softmax on random q, k, v across $T \in \{512, 2048, 8192\}$ (Dao *et al.*, 2023; Kwon *et al.*, 2023). Failures cluster at long contexts and low head dims (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Production gates require bounded-error tests before enabling fused paths in YiRage exports (Chen and YiRage Contributors, 2026; Chen, 2020).

Attention sinks and bias terms. Some models add attention sinks or ALiBi-style biases before softmax (Liu *et al.*, 2024). Online updates must incorporate biases into block-local max and exp before merge—a separate bias kernel between $QK^\top$ and softmax reintroduces Boundary A breaks (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

1.2.1 Causal masks, padding, and score biases

Decode normally makes the mask implicit: the current query attends only to KV positions written before it, so no upper-triangular filter exists inside the loop (Dao *et al.*, 2023; Kwon *et al.*, 2023). Masks become explicit in chunked prefill, when pages contain padding, when cross-attention reads encoder state, or when a model prepends a fixed sink token and must exclude filler rows (Liu *et al.*, 2024; Dao *et al.*, 2022).

Apply masked positions as $-\infty$ before the block-local max, never by subtracting a bias after *exp*. A post-hoc zero creates an apparent softmax mass that must be renormalized out later; a negative bias leaks a small nonzero weight into the accumulator; both corrupt (m, ℓ, o) in ways that grow with context length (Dao *et al.*, 2022; Chen, 2026). The carrier update itself does not change: the block-local max still drives the exponentials, and masked lanes simply never contribute to ℓ or o (Dao *et al.*, 2023; Davies *et al.*, 2025).

Padding rows deserve a compile-time decision, not a per-position runtime branch. When the number of real tokens is known per request, choose the block layout and page order so masked lanes collapse at block edges; when padding is scattered, mask flags must feed the same reduction tree as scores so no lane produces a separate softmax over filler positions (Chen and YiRage Contributors, 2026; Chen, 2020). Attention sinks are the intentional exception: treat them as regular keys whose bias participates in the block-local max, because the sink only shapes the output if its score flows through the same stable *exp* path as real positions (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026).

1.3 Cross-Hardware Online Softmax

Bandwidth-normalized comparison. Report online softmax wins as Δbytes/token and Δkernels/token, not only ms/token (Chen, 2026; Vellaisamy *et al.*, 2026). A CPU win of 5% latency with 40% bytes removed tells a different story than GPU ms shifts entangled with driver version (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

XDNA compile failures as signals. When XDNA rejects a schedule for carrier SRAM overflow, treat it as a fusion boundary violation to fix upstream—not as "NPU unsupported" (AMD, 2024a; Chen and YiRage Contributors, 2026). Successful GPU kernels that spill carriers are equally incorrect, just slower instead of failing compile (Chen, 2026, 2020).

Edge Ryzen AI deployment. DDR-contended edge hosts mirror CPU patterns: KV bandwidth dominates; online softmax prevents an extra P pass but cannot fix unfused MLP boundaries (AMD, 2024b; Davies *et al.*, 2025). Edge fleets should still measure launches/token even when "launch" means DMA edge count (AMD, 2024a; Vellaisamy *et al.*, 2026).

Same math, different residency tables. Online softmax is target-agnostic in equations but not in memory hierarchy (AMD, 2024b; NVIDIA, 2022; Goodfellow *et al.*, 2016). A schedule legal on Hopper may illegalize on XDNA when carrier slots exceed tile SRAM (AMD, 2024a; Chen and YiRage Contributors, 2026). CPU decode uses SIMD registers for (m, ℓ) and blocked o updates with cache-aware KV reads (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

GPU (Hopper-class). Warp-level reductions compute block max; **exp** uses fast math intrinsics with documented error bounds (NVIDIA, 2022; SemiAnalysis, 2024). TMA loads K/V blocks while MMA computes $QK^\top$ fragments (SemiAnalysis, 2024). Carriers stay in registers owned by the softmax warpgroup—not in HBM (Chen, 2026; Dao *et al.*, 2022). Flash-Decoding maps splits to SMs but must merge carriers without an extra launch when possible (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).

XDNA / AIE. Static dataflow assigns fixed SRAM addresses for (m, ℓ, o) for each pipeline stage (AMD, 2024a,b). Online updates become MAC/DMA micro-programs; there is no dynamic spill (Chen and YiRage Contributors, 2026; Chen, 2020). If softmax carriers cannot be placed, the graph fails compile—correct outcome (AMD, 2024a; Chen, 2026). Teams porting CUDA softmax often expect runtime allocation; XDNA requires upfront bucketization by context length (Kwon *et al.*, 2023; AMD, 2024b).

CPU. oneDNN/OpenBLAS-style paths vectorize **exp** and use FP32 accumulators (Goodfellow *et al.*, 2016). KV bandwidth dominates; online softmax savings are real but smaller than GPU HBM round-trip elimination (Davies *et al.*, 2025; Chen, 2026). NUMA-local KV still matters for multi-socket hosts (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 1.3.1. Identical Llama head at ctx=2048: Hopper fused online softmax keeps carriers in registers; XDNA maps them to tile slot #3; CPU holds m, ℓ in AVX registers—same invariants, different compiler legality checks (AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

Triplet regression. Never benchmark GPU only and infer XDNA behavior (AMD, 2024a; Davies *et al.*, 2025). YiRage triplet-test from one IR export should show bytes/token deltas, not just ms/token, so mechanism changes are visible across targets (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024).

Blackwell / next-gen SFU note. Newer GPUs continue to widen SFU throughput for softmax-heavy decode (NVIDIA, 2022; Davies *et al.*, 2025). Online softmax still wins on bytes moved; faster **exp** alone does not remove materialized P (Chen, 2026; Dao *et al.*, 2022). Codesign means updating tile sizes when SFU:MA ratios shift (SemiAnalysis, 2024).

1.4 Softmax Fusion Boundaries

Case study narrative (qualitative). Team A graph-captures unfused attention+MLP; Team B fuses online softmax with PV and keeps attn output in shared memory through SwiGLU (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Team A improves host time; Team B improves bytes/token (Chen, 2026; Davies *et al.*, 2025). Both claim "fusion"—only Team B advances mechanical sympathy (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Autotuner guardrails. Search algorithms that reward FLOPs or occupancy may break Boundary B (Chen, 2020; Chen and YiRage Contributors, 2026). Hard-constrain IR: no global stores on carrier SSA values; penalize P materialization with infinite cost (Chen, 2026; Vellaisamy *et al.*, 2026).

Serving engine integration. vLLM and TensorRT-LLM integrate flash-style attention but expose different fusion depth to custom compilers (Kwon *et al.*, 2023; Davies *et al.*, 2025). When exporting graphs to YiRage, preserve online regions through Boundaries A–B or explicitly document breaks for regression tests (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Serving engine integration contract. An engine that already calls a fused attention kernel still leaves an integration contract: which context buckets use the online path, which use eager fallback, and which page-table shapes are legal for the current carrier layout (Kwon *et al.*, 2023; Davies *et al.*, 2025). Record that boundary metadata at export time so a later framework upgrade cannot silently switch a decoder layer back to materialized softmax without a regression gate

Table 1.3: Fusion boundaries and decode metrics impact.

Boundary break	**Launches/token**	**Bytes/token**
Materialize P	+1 risk	High
Spill (m, ℓ, o)	0–1	Medium
Export attn out before MLP	+1	High
Full MegaKernel (Ch. 11)	$\leq$1/layer	Lowest

noticing (Chen and YiRage Contributors, 2026; Chen, 2026). The engine layer should be benchmarked end-to-end, but the bytes/token and kernels/token counters decide whether a given integration is really running the online path (Vellaisamy *et al.*, 2026; Chen, 2020).

Where online softmax ends. The **fuse** decision is not "softmax kernel + matmul kernel." It is: do (m, ℓ, o) and the PV accumulator survive into the O projection and MLP without HBM export (Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022)? Chapter 11's MegaKernel assumes the answer is yes (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Breaking fusion here forces separate launches and full-width tensors (NVIDIA, 2024; Kwon *et al.*, 2023).

Boundary A: after $QK^\top$, before PV. Fusing **online** softmax with PV inside the KV loop is the FlashAttention core (Dao *et al.*, 2022). Splitting materialized P to HBM between them negates IO savings (Chen, 2026).

Boundary B: after attention output, before MLP. Even perfect online softmax inside attention fails if attn output is written to HBM before SwiGLU (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). This is the dominant MegaKernel seam (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024).

Boundary C: debug stats export. Prefill microbenchmarks sometimes keep softmax stats for debugging—strip them in inference exports (Goodfellow *et al.*, 2016; NVIDIA, 2024). Accidental stat tensors reintroduce $O(T)$ HBM traffic (Chen, 2026).

Table 1.3 ties MegaKernel planning to TaxBreak counters (Vellaisamy *et al.*, 2026; Chen, 2026). Compiler **fuse** passes must label preserved boundaries in IR metadata (Chen and YiRage Contributors, 2026; Chen, 2020). Autotuners must

not "win" on FLOPs by breaking Boundary B (Davies *et al.*, 2025; SemiAnalysis, 2024).

Worked contrast. Hopper may keep carriers in registers through Boundary B with sufficient shared memory (SemiAnalysis, 2024; NVIDIA, 2022). XDNA requires static slot assignment for the same lifetime (AMD, 2024a). CPU may split Boundary B if L2 is exceeded—document the split explicitly in fleet configs (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

CUDA Graphs without residency. Memory-Floor shows CUDA Graphs on Qwen-2.5-7B yield 1.259× on H100 by removing host overhead yet leave HBM traffic unchanged (Chen, 2026; NVIDIA, 2024). Graph-capturing an unfused softmax+PV pair reduces launches but not Boundary A bytes—the Fregly-style distinction between orchestration relief and mechanical sympathy (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

IR metadata contract. YiRage should record `online_softmax_carriers` lifetime through Boundaries A–B so backends reject schedules that insert `GlobalStore` on P or attn output (Chen and YiRage Contributors, 2026; Chen, 2020). Code review diffs should show eliminated stores, not just fused kernel names (Chen, 2026; Vellaisamy *et al.*, 2026).

Industry context: DeepSeek MLA and Flash paths. DeepSeek's Multi-Head Latent Attention compresses KV and reshapes attention math for bandwidth-limited H800 deployments (Liu *et al.*, 2024; Davies *et al.*, 2025). Whether scores pass through a latent projection or full $QK^\top$, the softmax step still demands online carriers when decode avoids materialized P (Dao *et al.*, 2022). Treat MLA as a different QK producer feeding the same state machine in Figure 1.1—not an excuse to skip online updates (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Kernel launch accounting. Separating "stable exp" into its own kernel adds one launch per KV block in the worst case—multiplied by layer count and token steps (Vellaisamy *et al.*, 2026; NVIDIA, 2024). At 847 launches/token baseline for dense Llama-3.2-1B, even +1 launch/layer/block is measurable in p99 (Vellaisamy *et al.*, 2026; Chen, 2026). Online softmax belongs inside the same captured body as $QK^\top$ and PV (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Compiler lowering sketch. YiRage-style IR represents `OnlineSoftmax` as a region with explicit carrier SSA values live across `For` loops over KV blocks (Chen and YiRage Contributors, 2026; Chen, 2020). GPU lowering maps carriers to registers; XDNA lowering maps them to tile addresses; CPU lowering maps them to vector registers with spill forbidden by legality check (AMD, 2024a; Goodfellow *et al.*, 2016). Any pass that inserts `GlobalStore` on carriers must fail closed (Chen, 2026).

Warp-level implementation notes. A common GPU pattern assigns one warp group to max-reduce scores, another to scaled **exp**, and a third to accumulate o_t against V fragments (SemiAnalysis, 2024; NVIDIA, 2022). Shared-memory staging holds the current score tile; carriers live in registers owned by the softmax leader lane (Dao *et al.*, 2022). Bank conflicts on score tiles show up as p99 spikes at certain head dims—profile before declaring softmax "done" (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Roofline for softmax epilogue. Even when $QK^\top$ is bandwidth-bound, materializing P adds an extra read/write pass over T elements per head (Chen, 2026; Goodfellow *et al.*, 2016). Online softmax removes that pass; the remaining work is dominated by PV and KV streaming (Dao *et al.*, 2022, 2023). Optimizing **exp** alone without removing P is the attention analog of CUDA Graphs without residency relief (Chen, 2026; NVIDIA, 2024).

Long-context regression matrix. Stability tests should include: (i) random Gaussian q, k; (ii) structured spikes simulating attention sinks; (iii) FP8 with scales (Liu *et al.*, 2024; Davies *et al.*, 2025). Report max absolute error vs FP64 reference at $T = 8192$ and document dtype choices in Table 1.2 (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026). Fleet releases without this matrix risk silent quality drift in fused paths (Chen, 2020).

Disaggregated prefill/decode. When decode nodes run steady batch-1 steps, online softmax runs every token against a growing KV cache (Davies *et al.*, 2025; Kwon *et al.*, 2023). Prefill nodes may use different tile shapes; do not reuse prefill-only softmax kernels on decode pools without carrier residency review (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026). Disaggregation shifts optimization target toward per-step latency and in-kernel persistence (Chen, 2026).

Expert/MoE caveat. MoE decode adds routing launches unrelated to softmax—TaxBreak shows 9,305 launches/token for OLMoE-class models (Vellaisamy *et al.*, 2026). Online softmax still matters inside attention, but fleet diagnostics must separate routing tax from attention residency tax (Davies *et al.*, 2025; Liu *et al.*, 2024). Do not attribute MoE launch storms to softmax materialization without profiling (Vellaisamy *et al.*, 2026).

Tooling: what to profile. Use Nsight Compute warp stall reasons on softmax regions; PyTorch profiler op boundaries to count separate `softmax` ops; IR diff to verify carrier lifetimes (Davies *et al.*, 2025; Chen, 2020). Goodput improvement shows up as reduced DRAM bytes and fewer `softmax`/`exp` kernels—not higher tensor-core utilization (Chen, 2026; Vellaisamy *et al.*, 2026).

Anti-pattern: "FlashAttention enabled" checkbox. Enabling FlashAttention in a framework server removes many prefill pitfalls but may leave decode MLP/norm boundaries unfused (Dao *et al.*, 2022, 2023). Online softmax inside flash kernels is necessary, not sufficient, for Chapter 11 MegaKernels (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Benchmark bytes/token across the full decoder layer, not attention isolation (Chen, 2026; Davies *et al.*, 2025).

Handoff checklist to Chapter 11. Before merging MegaKernel work, verify: (1) carriers never stored globally between KV blocks; (2) Boundary B preserved in IR; (3) split-KV merge in same launch; (4) dtype/stability tests pass at max context bucket (Chen and YiRage Contributors, 2026; AMD, 2024a; SemiAnalysis, 2024). Failure at any item means Chapter 11 fusion is cosmetic graph capture (Chen, 2026; NVIDIA, 2024).

Quantitative anchor (TaxBreak). Dense Llama-3.2-1B at BS=4/SL=2048 averages 847.5 GPU kernel launches per *output* token on H100 (Vellaisamy *et al.*, 2026). Attention softmax epilogues are a fraction of those launches, but materializing P adds sync points that block fusion with PV and MLP (Chen, 2026; Dao *et al.*, 2022). Removing P is prerequisite to collapsing launches toward O(1) per layer—the Chapter 11 target (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; NVIDIA, 2024).

Quantitative anchor (Memory-Floor). CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259× on H100 by removing ≈3.05 ms host overhead yet

leaves device bytes unchanged (Chen, 2026; NVIDIA, 2024). Online softmax addresses device bytes; graphs address host launches—compose both, do not confuse either with MegaKernel residency (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Software stack placement. Online softmax sits below framework servers and above MMA/TMA microkernels (Kwon *et al.*, 2023; SemiAnalysis, 2024). Compilers own carrier lifetimes; hand-written CUDA owns tile sizes (Chen, 2020; Chen and YiRage Contributors, 2026). Performance engineers own regression matrices tying both to ms/token and goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Reader exercise (mental model). For ctx=4096, block=128, estimate KV blocks per head, carrier count per warp group, and whether materialized P adds one or two full DRAM passes (Dao *et al.*, 2022; Chen, 2026). If you cannot estimate bytes/token on a whiteboard, profiler tuning will misfire (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Safety and monitoring. Production fleets should log fused-path flags and context buckets per request (Kwon *et al.*, 2023; Davies *et al.*, 2025). Silent fallback to eager softmax when fusion legality fails must alarm—otherwise SLO regressions hide behind rare code paths (Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026).

Cross-chapter dependencies. Chapter 8 TMA double-buffering feeds KV tiles into this state machine (SemiAnalysis, 2024). Chapter 9 sync hierarchy ensures carrier merges see consistent V fragments (AMD, 2024a; Dao *et al.*, 2023). Chapter 11 assumes this chapter's invariants when fusing MLP (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Skipping any prerequisite yields "MegaKernels" that are launch wrappers (Chen, 2026; NVIDIA, 2024).

Glossary alignment. **Online softmax** = streaming softmax with (m, ℓ, o) carriers; **materialized softmax** = writing P to memory; **Boundary A/B** = fusion seams defined in Section 1.4 (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Use these terms consistently in design docs and IR comments (Chen, 2020; Chen and YiRage Contributors, 2026).

Cost framing. At cloud scale, wasted HBM bytes translate directly to rented GPU seconds (Davies *et al.*, 2025; Liu *et al.*, 2024). Online softmax is a cost

optimization—similar to Fregly's goodput ROI argument for performance engineers (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Small per-token byte savings compound across millions of tokens and hundreds of layers (Chen, 2026; Kwon *et al.*, 2023).

When online softmax is insufficient. If MLP and norm boundaries remain unfused, fixing softmax alone cannot hit aggressive ms/token targets (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Treat this chapter as necessary infrastructure, not the final decode optimization (Chen and YiRage Contributors, 2026; Dao *et al.*, 2023). The profile-driven engineer sequences work: residency first, launches second, FLOPs last (Chen, 2026; SemiAnalysis, 2024).

Documentation deliverables. Ship with each fused kernel: pseudocode of merge equations, dtype table, max context bucket, split-KV merge diagram, and triplet bytes/token table (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). This is the O'Reilly-style "tooling + methodology" pairing applied to compiler outputs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

1.5 Profiling Online Softmax in Production Stacks

Latency composition reminder. Per-token decode latency aggregates weight read, KV read, attention math, MLP, sampling, and host orchestration (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023). Online softmax attacks the attention math *memory* term and enables fusion that reduces orchestration (Dao *et al.*, 2022; Chen, 2026; Chen and YiRage Contributors, 2026). Expect partial ms/token impact if MLP boundaries remain unfused—consistent with Memory-Floor graph results (Chen, 2026; NVIDIA, 2024).

Book roadmap hook. Part V manual MegaKernels assume you can hold (m, ℓ, o) through Boundaries A–B (Vellaisamy *et al.*, 2026; Chen, 2020). Part VI YiRage passes automate carrier lifetime analysis (Chen and YiRage Contributors, 2026; AMD, 2024a). This chapter is the semantic foundation both parts reference (SemiAnalysis, 2024; Dao *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Chen, 2026; Kwon *et al.*, 2023; NVIDIA, 2024).

Profile-first workflow (Fregly-aligned). Start from user-visible ms/token at batch-1, then decompose with TaxBreak-style launch accounting and Memory-

Floor-style bandwidth floors (Vellaisamy *et al.*, 2026; Chen, 2026). If R_{floor} is low on H100, prioritize eliminating P materialization and Boundary B breaks before tuning **exp** microkernels (Chen, 2026; Davies *et al.*, 2025). If R_{floor} is high on L4, bytes/token from materialized softmax may still dominate—online carriers remain mandatory (Chen, 2026; Dao *et al.*, 2022).

Nsight Compute signals. High `memory_throughput` with low `sm__throughput` during softmax regions suggests P traffic or spill (Davies *et al.*, 2025; SemiAnalysis, 2024). High `warp_stall_math` on **exp** may justify SFU tuning *after* residency is fixed (NVIDIA, 2022). Treat profiler screenshots as release artifacts tied to IR hashes (Chen and YiRage Contributors, 2026; Chen, 2020).

1.5.1 Micro-reproduction harness

Production regressions arrive with many variables entangled, so the first step of any softmax investigation is to build a repro that isolates attention. The harness should pin the model and head layout, the context bucket, the page-table shape, the KV block size and split count, the carrier dtypes, and the driver and compiler IR hash (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Changing any one of those pins invalidates comparisons, which is why the config belongs beside the kernel rather than in a private experiment folder (Chen and YiRage Contributors, 2026; Chen, 2026).

Run three variants in the same graph body: eager SDPA, online softmax inside the KV loop without Boundary B fusion, and online softmax with the attention output carried through to the MLP seam (Dao *et al.*, 2022; Chen, 2020). Each variant must emit ms/token, kernels/token, and bytes/token under identical warm-up and context conditions so the three levers—latency, launches, and memory traffic—can be attributed independently (Chen, 2026; Vellaisamy *et al.*, 2026). When only the third variant moves all three counters, the fix is a residency fix; when only launches move, the fix is orchestration; when only latency moves without bytes, re-check the profiler configuration before celebrating (Davies *et al.*, 2025; Kwon *et al.*, 2023).

Keep the harness small enough to run in a CI job on every candidate kernel, and archive its config hash with the result table. A regression that takes an engineer a week to reproduce is a process failure; the same regression caught by a one-layer harness is a one-line revert (Chen and YiRage Contributors, 2026; Dao *et al.*, 2023).

Table 1.4: Diagnostic signals for online softmax regressions.

Signal	**Likely cause**	**Next action**
+1 kernel/layer/token	Split softmax epilogue	Fuse into KV loop
High DRAM R/W vs baseline	Materialized P or spill	IR diff carriers
Long-context drift only	FP16 ℓ underflow	Promote accum dtypes
GPU ok / XDNA fail compile	Carrier SRAM overflow	Reschedule tiles
Graph speedup without bytes win	Orchestration only	Pursue Boundary B

Framework-level visibility. PyTorch profiler timelines that show distinct `softmax` ops per layer per token indicate Boundary A breaks in eager stacks (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). torch.compile may fuse some epilogues but cannot invent on-chip carriers if the graph exports P (Chen, 2020; Davies *et al.*, 2025). Compare against a hand-written online reference kernel on identical shapes before trusting compiler fusion (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Regression gates for YiRage exports. Every backend export should attach: (i) bytes/token delta vs eager; (ii) kernels/token delta; (iii) max abs error vs FP64 at $T = 8192$; (iv) legality pass/fail on XDNA (Chen and YiRage Contributors, 2026; AMD, 2024a; Chen, 2026). This mirrors MLPerf-style reproducibility discipline adapted to compiler outputs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Teaching vs shipping paths. Educational notebooks often materialize P for clarity (Goodfellow *et al.*, 2016). Production IR must strip those tensors—accidental debug hooks reintroduce HBM traffic at scale (Chen, 2026; NVIDIA, 2024). Separate "debug softmax stats" behind compile flags, default off (Chen, 2020; Chen and YiRage Contributors, 2026).

Historical note. Online softmax predates LLM serving—streaming softmax appears in numerical libraries and early attention approximations (Goodfellow *et al.*, 2016; Dao *et al.*, 2022). FlashAttention's contribution is pairing online

updates with IO-aware tiling on GPUs (Dao *et al.*, 2022, 2023). This chapter's decode focus follows industrial shift toward batch-1 latency and MegaKernel fusion (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025).

Open problems. Extremely long contexts ($T > 128$K) stress ℓ dynamic range even in FP32 (Davies *et al.*, 2025; Liu *et al.*, 2024). Block-wise normalization tricks and periodic rescaling may enter carrier state—document any non-standard invariant before Chapter 11 assumes three scalars suffice (Chen and YiRage Contributors, 2026; Dao *et al.*, 2023). Research prototypes belong behind explicit IR ops, not silent kernel hacks (Chen, 2020).

Summary before takeaways. Sections 1.1–1.4 form a single story: streaming softmax math, stable numerics, target-specific residency, and fusion seams measured by TaxBreak and Memory-Floor counters (Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022; AMD, 2024b; Chen and YiRage Contributors, 2026). Section 1.5 ties those threads to profiler workflows familiar from OŔeilly-style systems books—start from user latency, decompose bottlenecks, verify bytes and launches, then tune kernels (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024).

1.6 Key Takeaways

Appendix cross-reference. Verified launch and bandwidth numbers for this chapter appear in Chapter 1 Table **??** and Appendix regression matrices (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025). When citing ms/token improvements here, always pair with kernels/token and bytes/token from the same run configuration (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016). Mixed sources invalidate cross-chapter comparisons (Chen, 2020).

Speculative decoding interaction. Speculative paths run draft models with additional forward passes (Davies *et al.*, 2025; Liu *et al.*, 2024). Online softmax in the target model remains unchanged in math; draft acceptance does not remove KV loop residency requirements (Dao *et al.*, 2022; Kwon *et al.*, 2023). Profile speculative stacks separately—draft overhead dominates some deployments before attention bytes do (Vellaisamy *et al.*, 2026; Chen, 2026).

Energy angle. HBM reads dominate decode energy (Chen, 2026; Davies *et al.*, 2025). Online softmax reduces unnecessary DRAM toggling—relevant for edge Ryzen AI and cloud TCO even when ms/token gains look modest (AMD, 2024b,a; Goodfellow *et al.*, 2016). Energy-aware fleets should log bytes/token alongside power counters (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

- **Stream, do not materialize.** Maintain the running statistics (m, ℓ, o) across KV blocks so the softmax probability matrix P is never written to HBM during decode (Dao *et al.*, 2022; Chen, 2026). Materializing P even as a thin $1 \times T$ vector per head per token is catastrophic once block boundaries export it, which is exactly the traffic online softmax removes (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).
- **Max before exp.** Stability is part of performance—unstable online paths force FP32 fallbacks that break fusion (Dao *et al.*, 2022; Davies *et al.*, 2025).
- **Measure attention goodput.** Track bytes/token and launches/token alongside ms/token; FLOPs mis-rank bottlenecks at batch-1 (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).
- **Mechanical sympathy across targets.** GPU registers, XDNA SRAM slots, and CPU SIMD accumulators implement the same invariants with different legality rules (AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).
- **Respect fusion boundaries.** Boundaries A–B determine MegaKernel legality; graphs alone do not fix Boundary breaks (Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026).
- **Split-KV needs in-kernel merge.** Flash-Decoding wins require second-stage online reduction without extra launches (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).
- **Profile-driven gates.** Long-context numerical tests and buffer accounting block merges—not microbench GEMM tweaks (Chen and YiRage Contributors, 2026; Chen, 2020).
- **Pair the metrics, as Fregly pairs goodput with utilization.** Always report kernels/token and bytes/token alongside ms/token on the same chart, because either number alone hides the other limiter at batch-1 (Vellaisamy *et al.*, 2026; Chen, 2026). A softmax change that improves one while quietly regressing the other is invisible unless the metrics are shown together (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

- **Reject graph-only wins.** Orchestration speedups without residency relief are partial fixes—label them explicitly in benchmark tables (NVIDIA, 2024; Chen, 2026).
- **Document dtype and context buckets.** Fleet configs must pin max context, block size, and carrier dtypes per compiled graph (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024a).

Closing integration note. Industrial decode optimization is a stack problem: online softmax fixes attention residency, MegaKernels fix layer residency, disaggregation fixes fleet residency (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). None replaces the others (Chen, 2026; Liu *et al.*, 2024). Performance engineers document which layer of the stack each PR addresses—avoid claiming end-to-end wins from softmax alone (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Vendor roadmap awareness. Faster SFUs, TMA, and FP4 tensor cores change *how* softmax is implemented, not *whether* carriers stay on-chip (NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025). Re-tile kernels each silicon generation; invariants in this chapter remain (Dao *et al.*, 2022; Chen, 2020). Mechanical sympathy evolves with hardware but rejects materialized P in decode (Chen, 2026; Vellaisamy *et al.*, 2026).

Collaboration boundaries. Model teams own architecture; systems teams own serving; compiler teams own IR legality (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Online softmax sits at the intersection—requires joint review when changing head dims, context limits, or dtypes (Liu *et al.*, 2024; Kwon *et al.*, 2023). Fregly's cross-team transparency principle applies: publish benchmark configs alongside kernel changes (Vellaisamy *et al.*, 2026; Chen, 2026).

Final sanity check. If your fused attention kernel writes a tensor named `attn_weights` or `probs` to global memory during decode, you are not running online softmax in the production sense (Dao *et al.*, 2022; Chen, 2026). Rename, refactor, or reject the merge (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

Execution checklist. Before closing a softmax fusion PR: confirm IR shows no `GlobalStore` on probabilities; run FP64 reference at $T = 8192$; capture Nsight

DRAM throughput delta; re-run TaxBreak launches/token on BS=1; validate XDNA compile legality or document intentional CPU/GPU-only scope (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; NVIDIA, 2024; Liu *et al.*, 2024; Dao *et al.*, 2023).

1.7 Conclusion

Online softmax is the decode attention state machine: streaming KV blocks update (m, ℓ, o) without materializing P (Dao *et al.*, 2022, 2023). Numerical discipline and multi-hardware residency tables determine whether compiler-generated schedules can fuse through Boundaries A and B (AMD, 2024a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). This is mechanical sympathy at attention granularity—the same design philosophy Fregly applies at cluster scale, translated into on-chip carrier lifetimes and TaxBreak-visible launch counts (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024).

Chapter 11 consumes this chapter wholesale: full-layer MegaKernels assume carriers and partial PV survive into MLP fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). If softmax spills here, no downstream graph capture recovers the bytes/token budget—the same way cluster-scale goodput cannot recover from idle GPUs masked by headline utilization (Chen, 2026; NVIDIA, 2024). Teams that skip online carriers often chase speculative decoding or wider GPUs first—reasonable for product roadmaps, but ineffective when bytes/token and launches/token remain bound by materialized softmax and Boundary B breaks (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). The disciplined sequence matches this book and Freglyś profile-first ethos: measure useful work, fix residency, then amortize launches, then tune FLOPs (Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Carry forward explicit IR boundary metadata, profiler screenshots, and triplet bytes/token tables in every release notes bundle, triplet regression across GPU/CPU/NPU, and long-context stability tests as non-negotiable release gates for any attention fusion touching decode paths (Chen, 2020; Kwon *et al.*, 2023).

Handoff. Online softmax carriers are the Part IV residency prerequisite for full-layer MegaKernel integration in Chapter 11 on GPU, CPU, and XDNA fleet targets alike across the book. (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2026; Davies *et al.*, 2025; Chen, 2020; SemiAnalysis, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; NVIDIA, 2024; Liu

et al., 2024; Dao *et al.*, 2023).

Metric reminder. Useful decode throughput excludes bytes spent materializing P, idle launch gaps, and debug tensors (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; SemiAnalysis, 2024; AMD, 2024a; NVIDIA, 2024; Liu *et al.*, 2024; Dao *et al.*, 2023).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 4, 5, 9, 10, 11, 13, 14, 15, 16, 17, 19, 21, 22, 23, 24

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 4, 5, 10, 17, 20, 21

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 13, 14, 15, 16, 17, 18, 20, 21, 23, 24

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 2, 3, 4, 5, 7, 8, 9, 10, 12, 13, 14, 16, 17, 19, 20, 21, 22, 23, 24

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 12, 14, 16, 17, 18, 20, 21, 22, 23, 24

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 1, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 20, 22, 23, 24

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 4, 5, 7, 8, 10, 11, 13, 14, 18, 22

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 6, 11, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 24

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2, 4, 5, 8, 10, 11, 13, 14, 15, 16, 17, 18, 20, 21, 22, 23, 24

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24